SwiftParcel: Building a Reliable ETA Prediction Service

Module 4 : Assignment 5

Abhijeet Kumar
Roll No.: DA24B001

1 Introduction

In this assignment, I worked on building and improving an ETA (Estimated Time of Arrival) prediction service for SwiftParcel. My objective was not limited to serving predictions from a trained machine learning model. I wanted to understand how such a model could be exposed as an API, evaluated before promotion, deployed with controlled traffic allocation, and protected through release checks.

I divided the implementation into four parts. Part A focused on building and benchmarking the prediction API. In Part B, I used MLflow to register model versions and implement a promotion gate. Part C introduced Ray Serve for scalable model serving and sticky canary routing. Finally, Part D added smoke tests and operational abort criteria.

Throughout the assignment, I focused on measuring actual behaviour rather than assuming that an implementation was working simply because the code executed successfully.

2 Part A: ETA Prediction API and Performance

2.1 Implementation

I built the prediction service using FastAPI. The service loads the ETA model bundle once when the application starts, rather than reloading it for every incoming request. I implemented the `/v1/models/eta:predict` endpoint for both individual and batch predictions, along with `/healthz` and `/metrics` endpoints.

I also added request ID handling, input validation, structured validation errors, and basic Prometheus-style request and latency metrics. For invalid inputs, the API returns HTTP 422, while unexpected server errors are handled separately.

2.2 Performance experiments

I performed load tests at different concurrency levels and compared the baseline implementation with batch prediction and multiple Uvicorn workers. The results helped me understand how concurrency affects throughput and tail latency.

Configuration	Concurrency	RPS	p95 (ms)	Errors
Baseline	8	802.2	14.7	0
Batch size 100	1	371.8	3.4	0
Two workers	4	1709.8	2.9	0
Two workers	8	1829.8	5.4	0
Two workers	16	1771.8	12.1	0

The two-worker configuration achieved the highest observed throughput of approximately 1829.8 requests per second at concurrency 8. This was about 2.28 times the baseline peak throughput. Increasing concurrency to 16 did not improve throughput further and increased p95 latency.

Batch prediction also showed the benefit of amortising request overhead. For a batch of 100 records, the measured p50 latency was 2.6 ms, equivalent to approximately 0.026 ms per row.

I also implemented and ran 16 API tests, all of which passed.

My observation: Increasing the number of workers improved throughput substantially in my experiments, but simply increasing concurrency was not always beneficial. Measuring both throughput and tail latency was therefore important.

3 Part B: MLflow Model Registry and Promotion Gate

3.1 Model registration

I used MLflow to manage two versions of the ETA prediction model. I implemented a custom PyFunc wrapper so that both models expose a consistent prediction interface. The wrapper returns the predicted ETA, rounded to one decimal place, together with the model version.

I registered both models under the name `eta-model`. Version 1 was assigned the `prod` alias, while version 2 was assigned the `candidate` alias. I logged the models with input examples, signatures, and pinned Python requirements. I used SQLite as the MLflow tracking backend.

Registered model	Version	Alias
eta-model	1	prod
eta-model	2	candidate

3.2 Promotion gate

I implemented a promotion function that compares production and candidate models using four measurements: overall MAE, rush-hour MAE, rainy-condition MAE, and candidate p95 prediction latency.

The default promotion policy requires at least 3% relative improvement in overall MAE, candidate p95 latency no greater than 10 ms, and no more than 5% regression in either the rush-hour or rainy-condition MAE.

I evaluated both models on 2,000 fresh evaluation records and measured candidate latency using individual predictions.

Metric	Production	Candidate
Overall MAE	11.9553	6.1049
Rush-hour MAE	19.6799	8.5795
Rain-condition MAE	11.6227	6.2853

The candidate achieved a 48.94% improvement in overall MAE. Its rush-hour and rainy-condition errors also decreased, and its measured p95 latency remained well below the 10 ms limit.

With the default 3% improvement requirement, the promotion gate returned `promote=True`. I then increased the required improvement to 60%. Since the candidate achieved only 48.94%, the gate correctly returned `promote=False`.

I wrote seven unit tests covering successful promotion, insufficient improvement, latency violations, slice regressions, exact threshold boundaries, and custom thresholds. All seven tests passed.

3.3 Shadow comparison

I started two independent MLflow serving instances: production on port 5101 and candidate on port 5102. I sent the same 200 fresh single-row requests to both servers and compared their predictions and response times.

Measurement	Result
Requests per server	200
Mean absolute prediction difference	9.4845 minutes
Predictions differing by more than 10 minutes	35%
Production p95 latency	3.509 ms
Candidate p95 latency	3.659 ms

This comparison helped me see that two models can have similar serving latency while producing meaningfully different predictions. I also accounted for the fact that changing an MLflow alias does not automatically replace a model already loaded in a running serving process. A controlled rolling deployment or an explicit in-process model swap is needed to update the served model safely.

4 Part C: Ray Serve and Sticky Canary Routing

4.1 Serving architecture

I used Ray Serve to deploy production and candidate ETA models as separate deployments. The production deployment uses two fixed replicas, while the candidate deployment can autoscale between one and three replicas.

Each replica loads its model bundle once during initialization. I used fractional CPU allocations so that the deployments and router could fit within the available two-CPU Ray environment.

I implemented a FastAPI ingress router that receives customer requests and forwards them to the selected model deployment. The router supports an `X-Beta: 1` header to force candidate routing for testing.

4.2 Deterministic customer routing

For canary allocation, I used BLAKE2b hashing on the customer ID. The resulting stable bucket determines whether a customer is assigned to production or candidate. This ensures that repeated requests from the same customer are routed consistently.

I chose this approach instead of Python's built-in `hash()` because its results can vary across processes due to hash randomization. Stable hashing makes customer assignment predictable across requests and identical redeployments.

4.3 Verification results

I implemented five verification tests using 1,000 distinct customer IDs and a 20% candidate traffic weight.

Verification	Observed result
Candidate traffic allocation	219/1000 (21.90%)
Repeated customer assignments	200/200 consistent
Assignments after identical redeployment	200/200 preserved
Previous candidate users moved back to production	0
Beta-header candidate routing	50/50 successful

All five verification tests passed. In particular, increasing the candidate weight from 20% to 50% did not move any previously assigned candidate customer back to production. This confirmed the monotonicity property of the routing strategy.

For autoscaling, I configured a target of five ongoing requests, a five-second upscale delay, a thirty-second downscale delay, and a maximum of three candidate replicas. The shorter upscale delay allows the candidate to respond to increased load, while the longer downscale delay reduces unnecessary replica changes.

5 Part D: Release Smoke Tests and Abort Criteria

5.1 Smoke testing

I implemented a command-line smoke-test utility that accepts the API URL, expected model version, and maximum permitted p95 latency. The test checks service health, model version, prediction sanity, distance monotonicity, invalid-input rejection, and response latency.

I ran the smoke test against the Part A API with the expected model version set to v1 and an 80 ms p95 latency limit.

Check	Result
Health endpoint	PASS
Expected model version	PASS
ETA sanity	PASS
Distance monotonicity	PASS
Invalid input rejection (HTTP 422)	PASS
p95 latency (latest unified run: 4.26 ms)	PASS

All six checks passed, with zero request errors in the latency test. The measured p95 latency was 4.26 ms, comfortably below the configured 80 ms threshold.

5.2 Abort criteria

I also documented operational conditions that should trigger a deployment abort or rollback. These are proposed release policies, not results from an automated production monitoring experiment.

- Roll back if p95 latency exceeds 80 ms for five consecutive minutes.
- Roll back if the HTTP 5xx error rate exceeds 1% over a rolling five-minute window with at least 500 requests.
- Stop promotion if candidate overall MAE is more than 5% worse than production, based on at least 500 labelled deliveries.
- Stop promotion if rush-hour or rainy-condition MAE is more than 10% worse, with at least 100 observations in the affected slice.

These criteria provide a clear basis for stopping a rollout before a model causes sustained degradation.

6 Conclusion, Findings and Limitations

Through this assignment, I built a complete workflow around an ETA prediction model, from API serving and performance testing to model registration, promotion decisions, canary deployment, and release validation.

The experiments showed me that model quality alone is not enough for a reliable ML service. A candidate can improve prediction accuracy but still needs to satisfy latency and slice-level requirements. Similarly, deployment needs stable routing and release checks to reduce operational risk.

My final verification included 16 API tests, 7 MLflow promotion tests, 5 Ray Serve verification tests, and 6 release smoke checks. All these tests passed in their respective runs.

There are also limitations to this implementation. The shadow comparison used 200 requests, and the evaluation data is generated rather than collected from a live delivery system. The abort criteria are documented policies and were not connected to an automated monitoring and rollback controller. The smoke test was run against the Part A API; it does not establish that every production deployment scenario has been tested.

Overall, this assignment helped me understand how to move beyond a working prediction model and build a more reliable, measurable, and controlled machine learning serving workflow.